

C# 程式語言結構

- **6-1: C# 程式檔案的結構**

C#程式的撰寫格式.....	6-1
撰寫主控台應用程式.....	6-4

- **6-2: 類別封裝的做法**

什麼是封裝(Encapsulation)?.....	6-5
存取修飾詞.....	6-6
封裝的做法.....	6-7

- **6-3: Partial Class**

什麼是Partial Class.....	6-8
使用Partial Class.....	6-9

Module 06

C#程式的撰寫格式-1

- 在C#中，名稱的大小寫視為相異。
 - A 與 a 在C#中視為不同。
- 同一範圍(Scope)內，名稱不可重複使用。
 - 例如：在相同類別中，不能有兩個成員（方法、變數、屬性）使用相同名稱。
 - 例如：在同一方法中，也不能宣告重複名稱的變數。
- 應避免寫出下列的程式碼：

```
class customer { /*...*/ }  
class Customer { /*...*/ }
```

C#程式的撰寫格式-2

- C#的程式註解：
 - 註解內的文字不會被執行。
 - 用於說明程式邏輯或提醒自己與他人。
 - 方便日後維護與閱讀。
 - 常用寫法：

//單行註解

/*

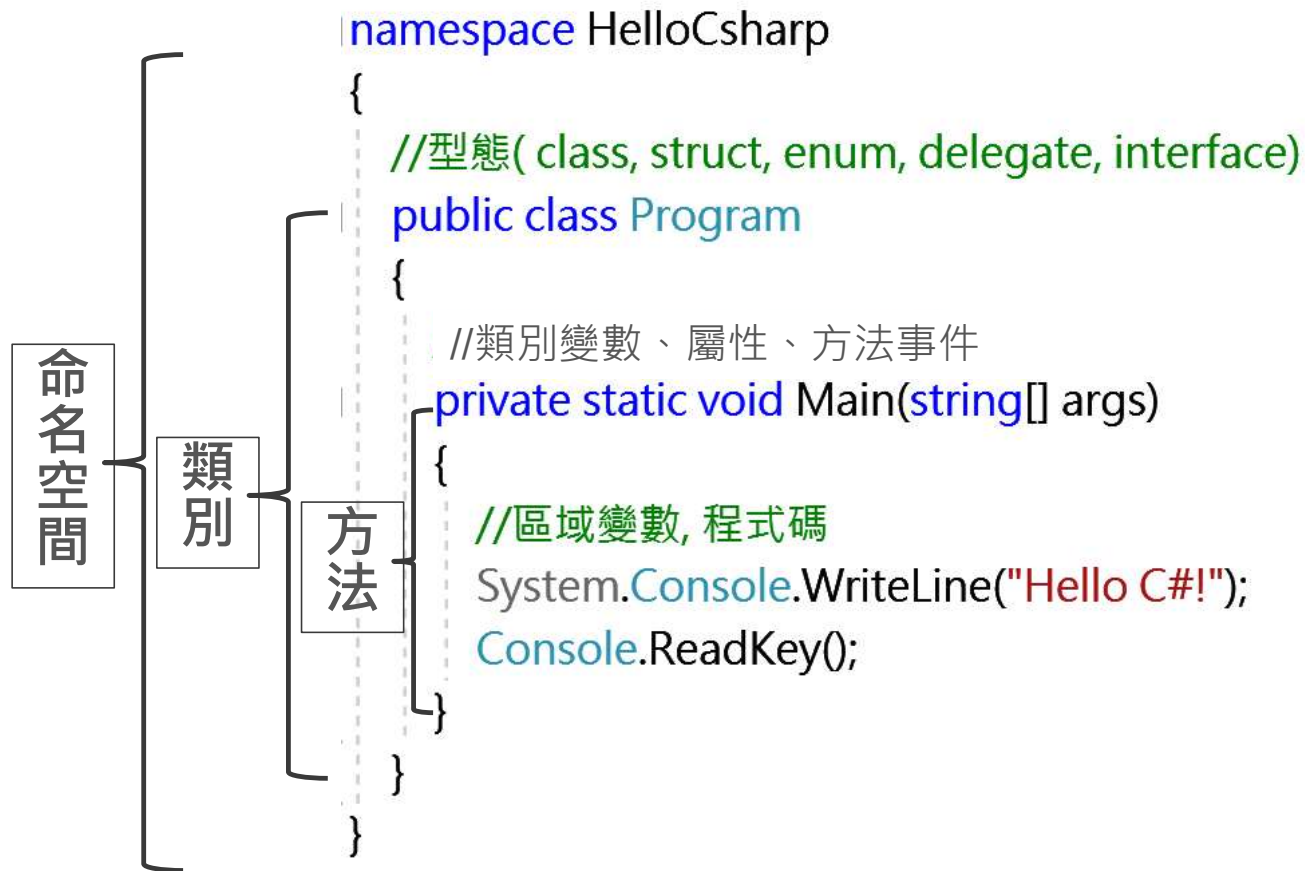
多行註解

行中註解

*/

C#程式的撰寫格式-3

引用命名空間 — `using System;`



撰寫主控台應用程式

```
using System;

namespace HelloCsharp
{
    public class Program
    {
        private static void Main(string[] args)
        {
            int x = 10;           //整數變數
            string str = "Hello C#!"; //字串變數
            Console.WriteLine(str);
            Console.WriteLine("10 + 10 :" + x*2);
            Console.ReadKey();
        }
    }
}
```

什麼是封裝(Encapsulation)?

- 是物件導向程式設計 (OOP) 的一個核心特性。
- 將物件的內部資料與實作細節隱藏起來。
- 外部只能透過物件提供的「公開介面 (方法/屬性) 」來存取。
- 保護物件內部資料，防止被隨意修改，保持物件完整性與正常運作。
 - 就像一般人無法隨意拆解汽車，只要了解如何操作 (方向盤、油門、煞車) ，就能開車，對車廠而言，封裝保護了汽車內部機件不被破壞。
- 封裝特性：
 - 隱藏實作細節：外部不需知道內部如何運作。
 - 控制存取範圍：透過存取修飾詞 (**private** 、 **public** 、 **protected** 等) 。
 - 提高安全性與穩定性：防止資料被不當修改。

存取修飾詞

存取修飾詞/屬性	說明
private	只有類別本身可以使用
protected	可被類別本身及子類別使用
public	任何地方都可使用
internal	同一組件(專案)中所有的類別都可使用
protected internal	可被同一組件的類別、類別本身及子類別使用

- 類別(class)：若未明確指定存取修飾詞，預設為 **internal**。
- 類別中的成員(屬性、方法等)：若未指定存取修飾詞，預設為 **private**。
- 介面(interface)中的成員：若未指定存取修飾詞，預設為 **public**。

封裝的做法

```
public class Program
{
    private static void Main(string[] args)
    {
        MyClass myclass = new MyClass();
        Console.WriteLine(myclass.str2);
        myclass.MyVoid();
        Console.ReadKey();
    }
}

public class MyClass
{
    private string str1 = "這是private屬性";
    public string str2 = "這是public";
    internal void MyVoid()
    {
        Console.WriteLine(str1);
    }
}
```

- **private** 屬性僅能在類別內部存取，外部無法直接修改。
- 可透過公開的屬性（**Property**）或方法，間接讀取或設定 **private** 欄位。
- 這樣的設計可保護資料完整性，避免外部任意更改物件內部狀態。

什麼是Partial Class

- 賦予類別跨越多個原始程式檔(.cs)共同撰寫同一類別。
- 只要Partial Class存在於相同namespace下，都視為同一類別。
- 若共同撰寫時沒加上partial關鍵字，則將出現重覆命名的錯誤。

使用Partial Class-1

- 在大型專案中，可使用**partial**關鍵字 將同一個類別拆分成多個檔案，方便由不同開發人員同時開發與維護。
- 若類別為系統自動產生的程式碼（例如設計工具生成），可透過 **Partial Class** 的方式進行擴充，而不需直接修改原始檔，以避免覆寫風險。

使用Partial Class-2

The image shows two side-by-side code editors in Visual Studio. The left editor, `ClsPartialSource1.cs*`, contains the following code:

```
1 namespace HelloCsharp
2 {
3     partial class MyPartialClass
4     {
5         string str1 = "Source1";
6     }
7 }
```

The right editor, `ClsPartialSource2.cs*`, contains the following code:

```
1 namespace HelloCsharp
2 {
3     partial class MyPartialClass
4     {
5         string str1 = "Source2";
6     }
7 }
```

An orange arrow points from the `str1` variable in the right editor to the error message in the bottom pane. The error message is:

錯誤清單
整個方案 1 錯誤 0 警告 0 訊息 組建 + IntelliSense
CS0102 類型 'MyPartialClass' 已包含 'str1' 的定義
專案 檔案 行 隱藏項目狀態
HelloCsharp ClsPartialSource2.cs 5 使用中

- 使用partial關鍵字表示編輯同一class，所以當變數名稱(str1)重複時會出現錯誤。